Continuous Tokens: Lossless Latent Byte Spans for Vocabulary-Free Transformers

Tamir Friedman[1,*] Daniel Alfasi[1] Netanel Moyal[1]

[1]Reco

tamirf@reco.ai daniela@reco.ai netanelm@reco.ai

*Corresponding author

August 2026

Abstract

Subword tokenization maps byte strings to discrete identifiers and then maps those identifiers to rows in large embedding and output matrices. We ask whether the identifier and the ordinary vocabulary are necessary at all. A *continuous token* is a non-empty variable-length byte span represented directly by a generated latent vector. A small local decoder certifies the exact payload by reconstructing the bytes and a private end marker; a one-byte path is always an exact fallback. The idea is deliberately modular. An input-only tokenizer can replace ordinary input lookup while preserving a frozen language-model backbone and native output head. An output-only tokenizer can map frozen hidden states to byte spans while retaining native input feedback. When composed, generated bytes are fed directly through the continuous input tokenizer, eliminating ordinary token IDs, the subword embedding table, and the vocabulary-sized output projection. Only byte primitives and a small structural-control interface remain discrete. We formalize the protocol, describe a compatibility bridge for existing models, derive its potential memory and compute advantages, and specify experiments that can falsify the proposal. Preliminary observations demonstrate native-head bypass and multi-byte macro-steps, but do not yet establish exact output generation or behavioral equivalence. We therefore present a modular architecture, analytical hypotheses, and a prospective empirical program rather than a claim of completed validation.

Keywords: tokenization, byte models, latent representations, sequence compression, language models.

1 Introduction

Transformers process vectors [7], yet most language-model interfaces insist on an intermediate discrete subword identity. A tokenizer first partitions text into members of a fixed vocabulary; an embedding table then turns each identity back into a vector. Generation reverses the pattern: a vocabulary-sized projection scores identities, which are finally decoded to bytes. This design is effective, but the global model inherits the vocabulary's fixed boundaries, parameter cost, language bias, and update cycle.

Byte-level models remove the vocabulary but increase the number of global positions. Patch and character models reduce that cost by pooling local symbols, but their latent units are generally internal, contextual, or not independently reversible. We explore a third point in the design space: a local neural tokenizer that maps an arbitrary candidate byte span to one continuous latent and accepts it only when a deterministic local decoder recovers the exact bytes.

The proposal has three deployment forms, illustrated in Figure 2:

- **Input-only:** replace ordinary input tokenization and lookup; retain the frozen backbone, native output head, native output tokenizer, and structural controls.
- **Output-only:** decode frozen final hidden states to non-empty byte spans or controls; retain the native input path for feedback.
- **Combined input-output:** compose both directions, feed generated bytes back through the continuous input tokenizer, and remove both ordinary vocabulary matrices.

Modularity is an advantage rather than a concession. Each direction can be tested, deployed, and rolled back independently. The combined system is the architectural end-state; the directional systems are useful products and controlled scientific ablations in their own right.

Our contributions are:

1. a codebook-free definition of a lossless continuous byte-span token;
2. an exhaustive segmentation rule that remains correct under non-monotonic validity;
3. modular input-only, output-only, and combined input-output Transformer interfaces;
4. a compatibility curriculum that intentionally overfits a frozen native embedding table;
5. an analytical account of vocabulary-state, KV-cache, context, and compute savings; and
6. a falsifiable evaluation protocol that separates exactness, behavior, density, and systems performance.

2 Background and Related Work

BPE and related subword methods address open-vocabulary language by composing frequent discrete units [4]. SentencePiece makes subword training language-independent and works directly from raw sentences [2]. These methods remain

Table 1: Positioning within the tokenization design space. "Independent inverse" means that one global unit can be decoded to its exact local payload without the original input sequence.

Method	Global unit	Fixed V	Exact	Retrofit
BPE / SentencePiece	token ID	**Yes**	**Yes**	**Yes**
ByT5 / CANINE	byte / character	No	Trivial	No
Charformer	learned block	No	No	No
BLT	contextual patch	No	No	No
Continuous tokens	latent vector	No	**Yes**	**Yes**

vocabulary based: each global position selects one row from a finite table.

Token-free models instead expose small alphabets to the network. ByT5 processes UTF-8 bytes and demonstrates robustness without a text tokenizer [8]; CANINE operates directly on characters while using downsampling for efficiency [1]. Their challenge is sequence expansion: a word represented by several bytes consumes several global positions. Charformer learns differentiable subword blocks through gradient-based tokenization [5]. BLT dynamically groups bytes into patches and allocates large-model computation according to local entropy [3]. These systems establish that local byte or character computation can support stronger global units.

Our proposal differs in two coupled requirements. First, a span is accepted only after exact, independent round-trip reconstruction. Second, the global unit is continuous and has no codebook identifier. VQ-VAE demonstrates the usefulness of learned discrete latent codes [6]; continuous tokens deliberately omit that quantization. The output Transformer need only produce a latent inside the decoder region corresponding to the desired byte span, not a particular codebook entry.

3 Continuous Token Protocol

Let the byte alphabet be $\mathcal{B} = \{0, \ldots, 255\}$ and let C be a small set of model protocol controls such as beginning-of-sequence, end-of-sequence, and role markers. Controls are structural events, not byte spellings. They bypass the byte tokenizer and preserve their exact semantics.

3.1 A reversible latent span

For a maximum local span length L, the input byte-span codec contains

$$E_\theta : \mathcal{B}^{1:L} \to \mathbb{R}^d, \tag{1}$$

$$D_\phi : \mathbb{R}^d \to (\mathcal{B} \cup \{\textsc{CodecEos}\})^{L+1}. \tag{2}$$

The decoder is parallel: learned output-position queries attend to one latent memory vector and produce 257-way logits. Greedy decoding terminates at the first private CodecEos. This symbol belongs only to the local codec; it has no input embedding and is unrelated to the language model's structural end-of-sequence control.

A candidate span s is valid exactly when

$$\text{valid}(s) \iff D_{\phi,0}(E_\theta(s)) = s \,\|\, \textsc{CodecEos}, \tag{3}$$

including the termination position. Every atomic byte bypasses the learned composition path and uses its exact frozen byte embedding. Atomic spans are therefore total, deterministic fallbacks.

The decoder partitions continuous space into implicit regions

$$\mathcal{R}_s = \{z \in \mathbb{R}^d : D_{\phi,0}(z) = s \,\|\, \textsc{CodecEos}\}. \tag{4}$$

There is no discrete codebook. For input, $E_\theta(s)$ must land inside $\mathcal{R}_s$. For output, a hidden-state adapter needs only to land anywhere inside the same region. Noise and margin training can enlarge useful regions around canonical encoder outputs.

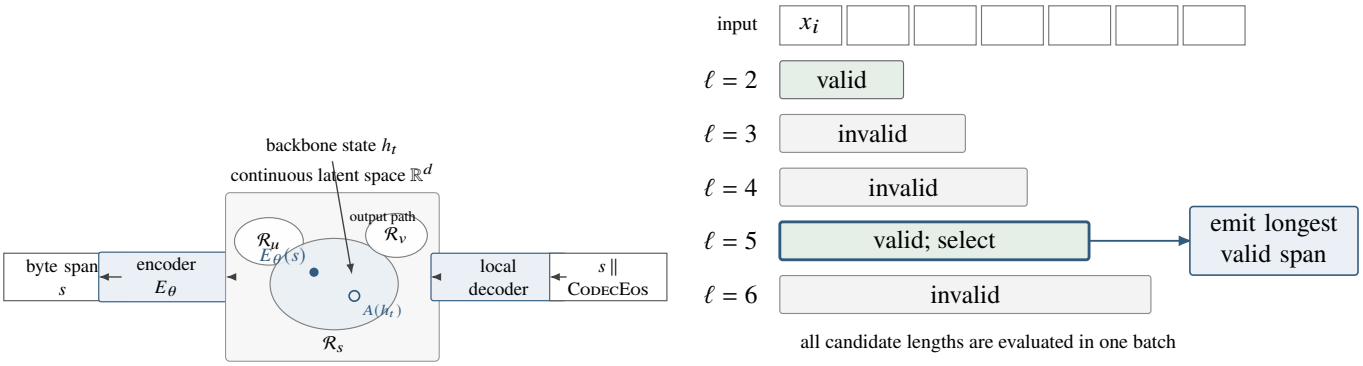
3.2 Exhaustive non-monotonic segmentation

Validity need not be monotonic in span length: a four-byte candidate may fail while a five-byte candidate succeeds. Consequently, a failure must never prune longer candidates. At byte offset i, all lengths are encoded and decoded in one padded batch, and the tokenizer selects

$$\ell_i^* = \max\left(\{1\} \cup \{\ell \in [2, L] : \text{valid}(x_{i:i+\ell})\}\right). \tag{5}$$

The default objective is longest-valid greedy segmentation. Dynamic programming can later replace the local objective without changing the codec protocol.

An encoding-only LRU cache may memoize $s \mapsto E_\theta(s)$ when evaluation is deterministic. It must return a bit-identical CPU copy, must not cache validity, and must never change candidate enumeration or acceptance. In the limit, a frozen cache of known spans behaves like a conventional embedding vocabulary, while the neural path supplies an open long tail.



(a) The decoder induces regions in continuous space. The input encoding $E_\theta(s)$ and an output adapter value $A(h_t)$ may differ numerically while decoding to the same exact span s.

(b) Validity is non-monotonic. Failure at lengths three and four does not prune the valid length-five candidate. If every multi-byte candidate fails, the exact atomic-byte path is used.

Figure 1: Two mechanics of continuous byte-span tokenization: codebook-free latent decoding regions and exhaustive longest-valid segmentation.

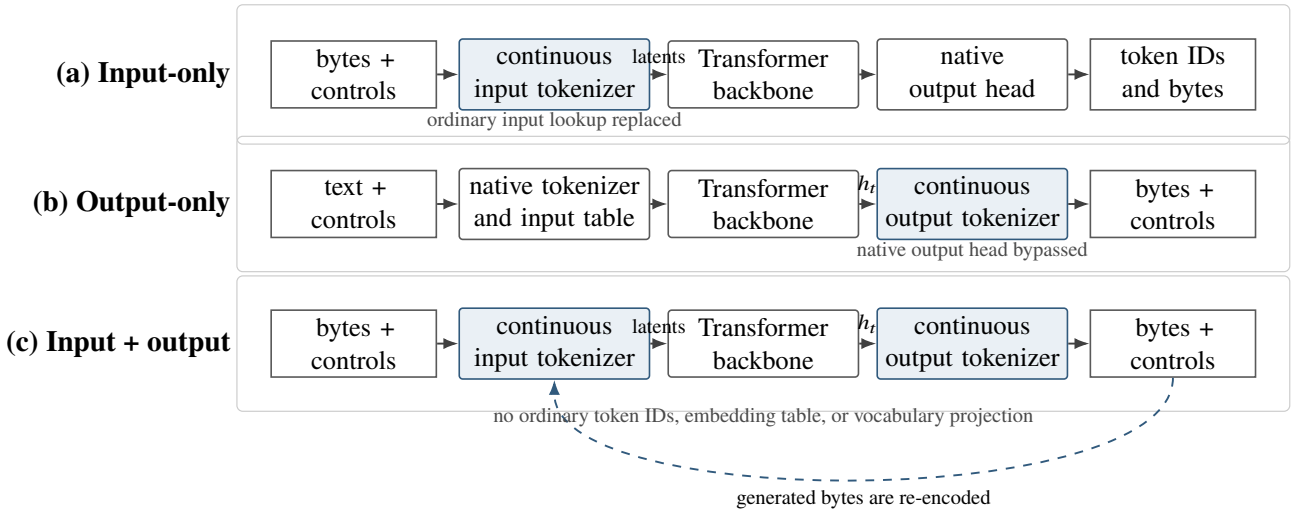


Figure 2: Three deployment forms of the same modular idea. Light blue boxes are learned continuous tokenizers; white boxes are retained native components. Input-only and output-only can be evaluated or deployed independently. Their composition closes the byte feedback loop and removes ordinary token IDs and both vocabulary-sized interfaces. Structural controls bypass the byte path.

4 Three Modular Deployment Forms

4.1 Input-only replacement

Let $W_{\text{in}} \in \mathbb{R}^{V \times d}$ be a frozen native input table. For an ordinary native token v with canonical byte payload s_v, define its compatibility target by

$$E_\theta(s_v) \approx W_{\text{in}}[v]. \tag{6}$$

Compatibility mode preserves native boundaries and changes only how ordinary embeddings are produced. Structural controls use their exact frozen rows. Dense mode then merges neighboring bytes whenever Equation 3 succeeds. The backbone consumes the resulting sequence via its ordinary continuous embedding interface.

This module is useful by itself: denser prompts can reduce global positions, KV-cache state, and prefill work while the native output distribution remains untouched. An untied input table can be physically omitted after replacement. A tied input/output table cannot be removed while the native output head still depends on it; that is an applicability limit, not a failure of input density.

4.2 Output-only generation

An output tokenizer maps the final hidden state h_t to either a structural control or a non-empty byte span. A parallel byte decoder predicts a finite sequence ending in its private termination symbol. In the standalone output-only form, emitted bytes are deterministically segmented into native token IDs for feedback to the unchanged input path. This permits a controlled experiment: the backbone and native feedback remain fixed while the vocabulary-sized output head is bypassed.

The output module can produce several bytes per global macro-step. Its central learning challenge is not reconstruction but conditional prediction: a long span must be both decodable and predictable from h_t. Sampling must also remain

meaningful when probabilities are defined over byte positions rather than a flat token vocabulary.

4.3 Combined input-output replacement

In the combined system, emitted bytes return through the continuous input tokenizer. The feedback loop contains no ordinary native token IDs. Structural controls remain explicit and small; ordinary language, programming-language text, binary content, and all text encodings share the same byte path.

This composition removes the ordinary input embedding matrix and the vocabulary-sized output projection, including the single shared matrix in a tied model. The global interface becomes conceptually smaller:

$$\begin{aligned}\text{bytes} &\to \text{local latent spans} \to \text{backbone} \\ &\to \text{local byte spans}.\end{aligned}$$

The claim is not that the complete network has no interface cost. It retains 256 byte primitives, structural controls, and local neural modules. The claim is that interface state and decisions no longer scale with an arbitrary vocabulary size V.

5 Training Strategy

5.1 Input curriculum

The native table is a finite supervised data set. Let $z_v = E_\theta(s_v)$. Stage 1 intentionally overfits every reachable, unambiguous ordinary row with

$$\begin{aligned}\mathcal{L}_{\text{in}} = \lambda_a \frac{\|z_v - W_{\text{in}}[v]\|_2^2}{\|W_{\text{in}}[v]\|_2^2 + \epsilon} \\ + \lambda_r \,\text{CE}\big(D_\phi(z_v), s_v \| \textsc{CodecEos}\big)\,.\end{aligned} \tag{7}$$

The 256 atomic byte rows and control rows are copied and frozen. Duplicate native IDs that decode to identical bytes but have different embeddings expose an information-theoretic conflict: one deterministic byte mapping cannot equal two targets. Such aliases must be reported, canonicalized, or retained structurally; they cannot be hidden by the loss.

Stage 2 freezes the selected aligned encoder and teaches its decoder arbitrary corpus and binary spans, replaying vocabulary payloads to preserve exactness. Stage 3, when needed, freezes every language-model parameter and distills native continuation behavior into dense input spans. Only the input tokenizer receives gradients.

5.2 Output curriculum

The output tokenizer trains on frozen backbone hidden states paired with the next byte span or control event. A practical curriculum begins with native-token payload boundaries, then grows to multi-token and arbitrary byte spans. Exact termination, byte accuracy, and feedback equivalence are optimized separately. Robustness noise around successful latent regions can make exact greedy decoding less brittle.

Table 2: Illustrative BF16 matrix state. "Byte rows" counts 256 model-width rows and excludes the local neural modules. MiB uses 2^{20} bytes.

V	d	One $V \times d$ matrix	256 byte rows
50,000	768	73.2 MiB	0.38 MiB
128,000	4,096	1,000 MiB	2.00 MiB
250,000	1,024	488.3 MiB	0.50 MiB

For an output decoder O_ψ, a simplified objective is

$$\begin{aligned}\mathcal{L}_{\text{out}} = \text{CE}\big(O_\psi(h_t), s_{t+1} \| \textsc{CodecEos}\big) \\ + \lambda_f \mathcal{L}_{\text{feedback}} + \lambda_c \mathcal{L}_{\text{control}}.\end{aligned} \tag{8}$$

In a frozen-backbone retrofit, only ψ and any small hidden-to-latent adapter train. In a future from-scratch model, both directional tokenizers and the backbone could be co-trained, but that is a different empirical claim.

5.3 Why the modules stay separable

Input and output tokenizers share a byte protocol but optimize different conditional problems. The input encoder compresses an observed span; the output decoder predicts an unobserved span. Separate checkpoints and gates therefore improve clarity. They may share byte embeddings or local features only after independent evidence shows that sharing helps rather than entangles the two objectives.

6 Potential Systems Advantages

6.1 Vocabulary state

Let p be bytes per parameter, d the model width, and V the ordinary vocabulary size. Native interface state is approximately

$$M_{\text{native}} = \begin{cases} pVd, & \text{tied input/output matrix,} \\ 2pVd, & \text{untied matrices.} \end{cases} \tag{9}$$

The combined interface instead requires

$$M_{\text{continuous}} \approx pP_{\text{local}} + p(256 + |C|)d, \tag{10}$$

where P_{local} counts input and output tokenizer parameters. The proposal saves memory only when the full deployed local state is smaller than the removed matrix state. Table 2 gives illustrative BF16 arithmetic before local-module cost; these are not measured model results.

6.2 Positions, KV cache, and context

Let ρ be the ratio of native positions to continuous positions for identical bytes. If $\rho > 1$, a prompt requiring T native positions requires approximately T/ρ continuous positions. KV-cache state is linear in global positions,

$$M_{\text{KV}} \propto T\, N_{\text{layers}}\, N_{\text{kv}}\, d_h\, p, \tag{11}$$

Table 3: Idealized global-work ratios for density ρ. Local tokenizer work, padding, kernels, and hardware utilization are excluded and must be measured end to end.

ρ	KV / FFN	Attention	Context gain
1.25	0.80	0.64	1.25×
1.50	0.67	0.44	1.50×
2.00	0.50	0.25	2.00×
3.00	0.33	0.11	3.00×

Table 4: Exploratory output-only observations. These measurements are diagnostic and do not constitute a supported combined-system result.

Measurement	Three-seed mean
Native vocabulary head bypass	yes
Bytes per global macro-step	10.96
Valid non-empty termination	99.79%
Byte accuracy	17.91%
Exact next-span fraction	0.0096%
Rollout byte agreement	5.19%
Output codec / native-head state	4.65%

so the idealized KV ratio is $1/\rho$. At fixed global context length, byte capacity grows by ρ. Dense prefill attention scales approximately as $1/\rho^2$ and per-position feed-forward work as $1/\rho$, before local-tokenizer overhead.

The local codec is not free. Candidate enumeration, exact decoding, and output-span generation can erase theoretical savings if implemented poorly. Batched candidates, bounded span lengths, source-dtype deployment, model-local compilation, and encoding memoization are engineering tools, not substitutes for end-to-end measurements.

7 Preliminary Observations

Preliminary experiments exercised both directional paths, but do not yet validate the complete architecture. An exploratory three-seed Qwen output-only study bypassed the native vocabulary head and emitted more than one byte per macro-step. It did not learn exact next spans or faithful rollouts (Table 4). These results establish feasibility of the directional interface while locating the central unsolved optimization problem.

An earlier input efficiency pilot also selected no feasible candidate: its best normalized embedding RMSE was 0.8995 against a prospectively registered 0.01 ceiling. That outcome does not prove finite-table overfitting impossible; it shows that the tested architecture, capacity, budget, and objective did not solve it. The next study must isolate approximation capacity from reconstruction and density by first overfitting a fixed vocabulary subset, scaling capacity while remaining below the removable matrix budget, and only then restoring the full objective.

The important distinction is between *protocol verification* and *empirical evidence*. Controlled synthetic tests can establish exact fallback, exhaustive non-monotonic segmentation, cache neutrality, and frozen-backbone wiring. Only completed real-model experiments can support embedding compatibility, behavioral preservation, density, or measured savings.

8 Evaluation Protocol

The proposal should be accepted or rejected claim by claim, not by a single celebratory score. All thresholds, revisions, seeds, and held-out sets should be fixed before final runs.

Input compatibility. Measure normalized RMSE, cosine distribution, full-table retrieval, source-dtype bit equality, and exact byte reconstruction for every unambiguous ordinary native payload. Compare native and compatibility paths on KL and Jensen-Shannon divergence, top-k agreement, NLL, perplexity, and greedy generation.

Input density. Measure exact round-trip, bytes per continuous position, native-to-continuous position ratio, fallback rate, span-length histogram, and invalid candidates by length on held-out natural language, multilingual text, code, UTF-8/16/32 fixtures, and arbitrary binary bytes. The headline must require 100% round-trip.

Output quality and density. Measure exact next-span and byte accuracy, proper termination, control accuracy, native-feedback equivalence, bytes per macro-step, rollout edit similarity, likelihood, calibration, and sampling diversity. Output density counts only exact emitted bytes.

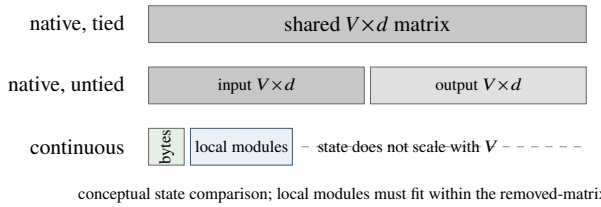
Combined-system behavior. Close the continuous feedback loop and compare sequence likelihood and generation behavior against the native model under identical prompts. Verify that no ordinary vocabulary tensor is loaded or serialized. For tied models, this is the first configuration in which complete removal is applicable.

Systems performance. In isolated processes, report serialized and resident state, peak memory, tokenizer preparation, prefill, time to first byte, decode latency, KV bytes, and energy when available. Run cache-disabled, cold-cache, and warm-cache conditions. Include all local-module computation and cache memory.

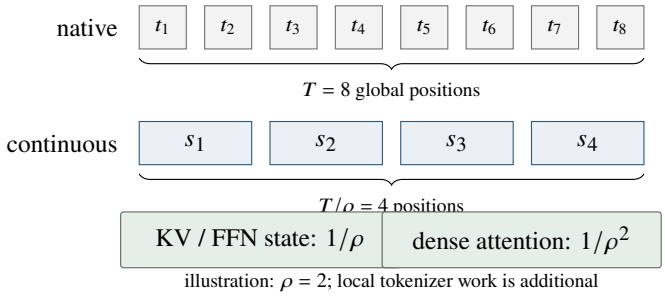
Replication. Use at least three fixed seeds and two tokenizer families. Publish every failed trial and preserve immutable manifests. A result may be operationally complete while scientifically unsupported.

9 Limitations and Open Problems

Predictability is not reconstructability. A codec may reconstruct a long random identifier from its observed bytes,



(a) Ordinary native interface state scales with vocabulary size V. In the combined system, only byte/control rows and local tokenizer parameters remain at the interface.



(b) For identical source bytes, denser spans shorten the global sequence. KV and per-position work scale linearly in position count; dense prefill attention has a quadratic sequence term.

Figure 3: Analytical sources of potential memory and compute savings. These diagrams show scaling relationships, not measured end-to-end speedups.

while an output model cannot predict that identifier from context in one macro-step. Span selection may ultimately need a rate or surprise criterion in addition to local validity.

Continuous generation needs a probability model. Greedy region membership is enough for deterministic decoding, but high-quality sampling requires calibrated probabilities over variable-length byte sequences. Parallel output positions are efficient but may model intra-span dependence less naturally than an autoregressive local decoder.

Finite precision bounds capacity. A finite-width, finite-precision latent cannot injectively represent arbitrary unbounded byte strings. Bounded spans and atomic fallback make the protocol total; they do not make compression free.

Native aliases can be contradictory. If two ordinary vocabulary rows map to identical canonical bytes but have different embeddings, a deterministic encoder cannot reproduce both. Compatibility claims must be restricted to a canonical reachable mapping or preserve the distinction structurally.

Local overhead can dominate. The combined architecture removes large vocabulary interfaces but adds local attention, decoding, verification, and segmentation. It is simpler in discrete semantics and asymptotic vocabulary state, not automatically in latency or implementation complexity.

Frozen-model replacement is the hard setting. An existing backbone was trained around native boundaries. Exact row imitation is a strong bridge, but merged spans still change position count and internal computation. A model trained from scratch with continuous tokens may realize more of the idea's potential, yet would provide weaker evidence that the method is a drop-in replacement.

10 Research Outlook

Continuous tokens suggest a model whose vocabulary is procedural rather than enumerated. Frequent spans may live in an ordinary memoization cache; rare or novel spans are composed by the same local network. The cache accelerates the model without defining what it can represent. Languages, encodings, code, and arbitrary binary streams share one protocol. New terms need no vocabulary resize, and a fixed global context budget can represent more source bytes where the local codec is effective.

The modular path makes the research tractable. Input-only experiments ask whether observed bytes can replace fixed lookup and compress context. Output-only experiments ask whether hidden states can predict exact byte spans and bypass a vocabulary head. The combined experiment asks whether the two independently validated modules can close the loop and eliminate ordinary vocabulary state. Each answer is useful, including a negative one.

11 Conclusion

The Transformer does not intrinsically require token IDs; it requires a sequence of vectors and a well-defined output distribution. Continuous byte-span tokenizers move discrete structure to the smallest universal boundary, retain exactness through local reconstruction, and allow the global sequence to become denser. Their strongest form replaces both ordinary vocabulary matrices with small directional byte modules around the backbone. Their most practical strength is modularity: input-only and output-only adoption are independently meaningful, while composition unlocks the full vocabulary-free architecture.

The proposal remains ambitious. Preliminary experiments establish execution paths and expose the hard problems, not the final scientific result. That is precisely why the idea is valuable as a research program: its claims are concrete, its failure modes are visible, and its potential benefits span model state, context length, KV cache, global compute, multilinguality, and architectural clarity.

References

[1] Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022. doi: 10.1162/tacl_a_00448.

[2] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018. doi: 10.18653/v1/D18-2012.

[3] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024. URL https://arxiv.org/abs/2412.09871.

[4] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016. doi: 10.18653/v1/P16-1162.

[5] Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2106.12672.

[6] Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. *Advances in Neural Information Processing Systems*, 30, 2017. URL https://arxiv.org/abs/1711.00937.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017. URL https://arxiv.org/abs/1706.03762.

[8] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. doi: 10.1162/tacl_a_00461.